ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



F.CSM302 Үйлдлийн систем · Бие даалт 1

Судалгаа, хэрэгжилт, туршилтын тайлан

Kernel Data Structures

Хуваарилалтын код: **4** · Үзүүлэлт: Kernel structure graph · Бүлэг 3 – Processes

GitHub: github.com/grl059/OS_BD1

Хичээл заасан багш: Г. Ганбат (F.CS10)

Гүйцэтгэсэн баг: Н. Ням-Од (B222270805)
А. Гэрэл (B241940001)
А. Номин (B242270088)
Э. Мягмаршагай (B241910001)
О. Хүслэн (B241910020)
Ч. Түмэншагай (B160910830)

2026–2027 оны хичээлийн жилийн намар · Семинарын цаг: 5-1 пар
Улаанбаатар

Агуулга

Хураангуй 3

1. Удиртгал 3

2. Сурах бичгийн шаардлага 3

3. Онолын үндэс 4

4. Системийн зохиомж 5

5. Хэрэгжилт 6

6. Туршилт ба UI дүрслэл 7

7. Туршилтын арга 10

8. Үр дүн 11

9. Хэлэлцүүлэг 14

10. Дүгнэлт 15

Ашигласан материал 15

Хавсралт А. Ажлын төлөвлөгөө ба гишүүдийн оролцоо 16

Хураангуй

Асуудал. Kernel процесс, нөөц, файлуудаа холбоост жагсаалт, дараалал, стек, bitmap, хэш хүснэгт, мод зэрэг өгөгдлийн бүтцээр удирддаг. Эдгээрт элемент нэмэх, устгах, хайх үед бүтэц хэрхэн өөрчлөгддөгийг нүдээр харуулж, хэрэгжилтийг нь нотлох шаардлагатай.

Шийдэл. Linux-ийн `list_head`-ийн next/prev семантикийг C-ээр дахин бичиж, долоон бүтэц (жагсаалт, queue, stack, bitmap, hash map, BST/AVL мод, PCB бүхий ready queue ба FCFS/Round Robin scheduler) бүхий командын интерпретатор (`kds`) бүтээв. Түүнтэй яг ижил дүрэмтэй хөдөлгүүр бүхий interactive UI (Kernel structure graph) нь үйлдэл бүрийн дараах бүтцийг граф хэлбэрээр шууд зурна. Сурах бичгийн 3-р бүлгийн Programming Project-ийн дагуу kernel module нь `struct color` жагсаалт болон `start` параметртэй Collatz дарааллын жагсаалтыг kernel орчинд үүсгэж, устгана.

Туршилт. 6 жишээ скрипт ба 5000×8 командын stress тест ASan/UBSan-тай давсан. C ба UI-ийн хөдөлгүүр 6 скрипт болон 40 000 санамсаргүй командад (8 seed × 5000, тэр дотор AVL) мөр мөрөөрөө ижил гаралт өгсөн. AVL-ийн өндрийн хязгаарыг 30 санамсаргүй гүйлтээр шалгасан. Kernel module-ийн өмнөх хувилбар Ubuntu 24.04 (kernel 6.11.0-17) дээр insmod/rmmod хийгдсэн.

Гол үр дүн. Оролт, огноо, хязгаарыг шалгадаг, хэрэглэгч өөр өгөгдлөөр дахин туршиж болох, CLI ба график UI хоёулаа ижил гаралт өгдөг хэрэгсэл бүтээгдсэн.

1. Удиртгал

Үйлдлийн системийн kernel нь процессын жагсаалт, ready queue, bitmap (нөөц чөлөөтэй эсэх), хэш хүснэгт (хурдан хайлт), модон бүтэц (scheduler, санах ой) зэргийг өгөгдлийн бүтцээр удирддаг. Төслийн **зорилго** нь эдгээрийн үндсэн үйлдлийг хэрэгжүүлж, үйлдэл бүрийн дараах төлөвийг нүдэнд харагдахуйц болгох явдал юм.

Ач холбогдол. `list_head` бол Linux kernel-ийн хамгийн их хэрэглэгддэг бүтэц бөгөөд `task_struct` (PCB), ready queue зэрэг нь түүн дээр суурилдаг. Үүнийг ойлгох нь 3-р бүлгийн «Processes» сэдвийн суурь болно.

Хамрах хүрээ. Хэрэглэгчийн орчны загвар (C, CLI), түүнтэй ижил хөдөлгүүртэй интерактив UI, мөн нэг kernel module. Бодит Linux scheduler-т холбогдохгүй.

2. Сурах бичгийн шаардлага

Хуваарилалт: код 4, «Kernel Data Structures», үзүүлэлт «Kernel structure graph», бүлэг 3 – Processes. **Хүрэх үр дүн:** «Сурах бичигт заасан kernel data structure-ийн хэрэглээг хэрэгжүүлж, элемент нэмэх, устгах, хайх үйлдлийн дараах бүтцийг графаар харуулна.»

Operating System Concepts-ийн 3-р бүлгийн Programming Project «Kernel Data Structures» нь 1.9-д тайлбарласан бүтцүүд дээр тулгуурласан хоёр хэсэгтэй:

- **Part I:** `struct color {red, blue, green; struct list_head list;}` төрлийн элементүүдийг kernel module дотор жагсаалтад нэмж, `list_for_each_entry`-ээр гүйлгэн хэвлэх, module-ийг буулгахад `list_for_each_entry_safe`-ээр устгаж санах ойг чөлөөлөх.
- **Part II:** module-д `start` параметр дамжуулж (`module_param`), Collatz дарааллыг (тэгш бол n/2, сондгой бол 3n+1, 1 хүртэл) холбоост жагсаалтад хадгалан хэвлэх, буулгахад устгах.

Доорх хүснэгт нь шаардлага бүрийг хэрэгжилт ба баталгаажуулалттай холбосон.

Шаардлага	**Хэрэгжилт**	**Нэмэх / устгах / хайх**	**Баталгаа**
Ch.3 Part I: struct color жагсаалт	kds_module.c, `color_list`	list_add_tail / list_del / list_for_each_entry	run_module_test.sh, dmesg
Ch.3 Part II: Collatz, `start` параметр	kds_module.c, `module_param(start)`, 1..1 000 000	list_add_tail / list_del	start=25, 15, 0 (татгалзана)
1.9.1 Холбоост жагсаалт (list_head)	Давхар холбоост дугуй жагсаалт (struct birthday)	add_head, add_tail / del / find	textbook, search, UI «Галт тэрэг»
1.9.1 Stack (LIFO)	list_head, дээд талаас нэмж авна	push / pop / find	all_structures, search
1.9.1 Queue (FIFO)	list_head, төгсгөлд нэмж эхнээс авна	push / pop / find	all_structures, search
1.9.2 Binary search tree	Энгийн BST (хамгийн муу O(n))	insert / delete / find	search: 1–5 → өндөр 5
1.9.2 Тэнцвэржсэн мод (O(lg n))	AVL (`tree mode avl`)	insert / delete / find	search, avl_check: 1–7 → өндөр 3
1.9.3 Хэш функц ба hash map	Кодын нийлбэр mod n; collision-д bucket доторх list_head	put / del / get	search: bob, ab, ba нэг bucket
1.9.4 Bitmap	n = 1..128 бит	set / clear / test	all_structures, errors
«Processes» холбоос	PCB + list_head ready queue, FCFS, Round Robin	proc new / дуусах / sched print	scheduler, scheduler_full

Харуулах шаардлага: UI нь бүтэц бүрийн нэмэх, устгах, хайх үйлдлийн дараах төлөвийг граф (вагон, дараалал, овоолго, нүд, bucket, мод, ready queue) хэлбэрээр шууд зурдаг (6-р хэсэг).

3. Онолын үндэс

list_head нь intrusive, давхар холбоост, дугуй жагсаалт юм. Node нь өгөгдлийн struct дотор `list_head` талбар агуулж, `container_of` макро нь талбарын хаягаас эзэн struct-ийг сэргээнэ.

Нэмэх, устгах үйлдэл O(1), хайх нь O(n).

Queue (FIFO) нь tail-д нэмж head-ээс авдаг. Kernel-ийн ready queue ийм зарчмаар ажилладаг. **Stack (LIFO)** нь head-д нэмж head-ээс авдаг.

Bitmap-д 1 бит нь 1 нөөцийг илэрхийлнэ. set/clear/test нь O(1) бөгөөд Boolean массиваас 8 дахин бага зай эзэлнэ.

Binary search tree-д зүүн < үндэс < баруун байна. Дундаж хайлт O(lg n), харин эрэмбэлсэн оролтод мод нэг шугам болж хамгийн муу тохиолдол O(n) болно. Тэнцвэржсэн мод n элементэд ойролцоогоор lg n түвшинтэй байдаг. Linux red-black мод ашигладаг бол бид ижил зорилготой AVL модоор (өндрийн зөрүү ≤ 1, эргүүлэлт) загварчилсан.

Hash map-д хэш функц түлхүүрийг хүснэгтийн индекс болгож, ихэнх тохиолдолд O(1) хайлт өгнө. Хоёр түлхүүр нэг индекст таарвал (collision) тухайн bucket-д жагсаалт залгана. Collision олшрох тусам хайлт O(n) руу ойртоно.

Процесс ба scheduler. Процесс бүр PCB-тэй байдаг (Linux-д `task_struct`). Бэлэн процессууд ready queue-д зогсоно. FCFS нь процессыг дуустал нь ажиллуулна. Round Robin нь процесс бүрд квант хугацаа өгч, дуусаагүй бол дарааллын төгсгөлд буцаана. Хүлээлтийн хугацааг нийт үргэлжилсэн хугацаанаас ажлын хугацааг хасаж олно.

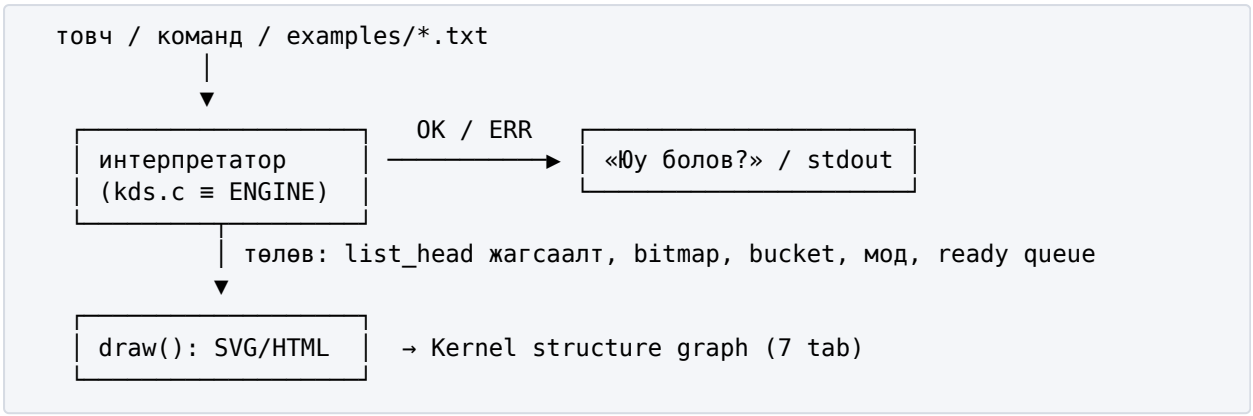
Kernel module ба параметр. `module_param(name, type, perm)` нь `insmod kds_module.ko start=15` хэлбэрээр утга дамжуулах боломж олгоно. Init амжилтгүй болбол `module_exit` дуудагдахгүй тул init өөрөө нөөцөө чөлөөлөх ёстой. Нийтлэг алдаанууд: `list_del`-ийн дараа pointer ашиглах, node чөлөөлөхгүй үлдээх (leak), гүйлгэж байхдаа устгах (энэ үед `_safe` хувилбар хэрэгтэй).

4. Системийн зохиомж

Компонентууд:

- **(a) src/user/kds.c** – командын интерпретатор ба 7 бүтэц. Жагсаалт, queue, stack, hash-ийн bucket, ready queue бүгд нэг `list_head` хэрэгжилтийг ашиглана.
- **(b) docs/ui/index.html** – ижил команд, мессеж бүхий JavaScript хөдөлгүүр (ENGINE), түүнээс граф зурдаг UI хэсэг (HTML/SVG).
- **(c) src/kernel/kds_module.c** – kernel орчинд ажиллах хувилбар: color, Collatz, birthday жагсаалт, kfifo, bitmap.
- **(d) tests/** – C ба UI-г хооронд нь, мөн хүлээгдэх гаралттай харьцуулдаг тестүүд.

Өгөгдлийн урсгал: хэрэглэгчийн команд (товч, командын мөр эсвэл script) → интерпретатор → бүтцийн төлөв шинэчлэгдэнэ → мессеж (OK/ERR) → (UI дээр) төлөвөөс график зурагдана. График нь тусдаа өгөгдөл биш, бүтцийн төлөвөөс шууд зурагддаг тул бодит ажиллагаатай үргэлж нийцнэ.



5. Хэрэгжилт

Хэл ба хэрэгсэл: C (gcc, `-Wall -Wextra -fsanitize=address,undefined`), JavaScript (HTML/SVG, сан ашиглаагүй), kernel C (Kbuild), туршилтад Python 3 ба Node.js.

Гол шийдлүүд:

1. `list.h`-ийн `__list_add`, `list_add`, `list_add_tail`, `list_del`, `list_for_each(_safe)`, `container_of`-ийг өөрсдөө бичсэн.
2. Queue ба stack нь ижил list_head-ийг хуваалцах бөгөөд зөвхөн нэмэх талаараа ялгарна. Багтаамж нь 8.
3. Огноо шалгахдаа өндөр жилийг тооцно: 4-т хуваагддаг ба 100-д хуваагддаггүй, эсвэл 400-д хуваагддаг.
4. Модноос хоёр хүүтэй node устгахдаа баруун дэд модны хамгийн бага утгаар орлуулна. AVL горимд буцах замдаа `bal()` нь rotl/rotr эргүүлэлтээр тэнцвэржүүлнэ. Модны төрлийг зөвхөн хоосон үед солино.
5. Hash map нь 1–16 bucket-тэй. Хэш функц нь тэмдэгтүүдийн кодын нийлбэр mod n. Collision-ийг bucket доторх list_head жагсаалтаар шийднэ (chaining). Нийт 24 элементээр хязгаарласан. `get` үйлдэл хайлтын алхмын тоог мэдээлнэ.
6. PCB нь `struct pcb {pid, name, rem, slice, list_head}` бүтэцтэй. Ready queue-д `list_add_tail`-аар нэмж, эхний элементийг `list_del`-ээр авч CPU-д өгнө. `sched tick` бүрд CPU 1 тик ажиллана. FCFS нь процессыг дуустал ажиллуулна. RR-д квант дуусвал процесс дарааллын төгсгөлд буцна (preempt). Амьд процессын тоо 8 хүртэл.
7. Хайх үйлдэл бүх бүтцэд байна: list find (индекс), queue/stack find (байрлал), tree find, bitmap test, hash get.
8. **Kernel module:** init дотор color (4 элемент), Collatz (start-аас 1 хүртэл, хамгаалалтын хязгаар 1000 гишүүн), birthday, kfifo, bitmap-ийг үүсгэнэ. `start` нь 1..1 000 000 мужаас гадуур бол `-EINVAL` буцааж module ачаалагдахгүй. `kmalloc` алдаа гарвал `goto fail`-аар бүх жагсаалтыг цэвэрлэнэ. Exit дотор `list_for_each_entry_safe`-ээр элемент бүрийг хэвлэж устгана.
9. C ба UI хоёр тусдаа бичигдсэн боловч бүх команд, мессеж, хязгаар нь ижил. Тоон аргументыг хоёулаа `[+-]?цифр` хэлбэртэй, int мужид багтах ёстой гэж шалгадаг тул

гаралт нь яг таарна.

5.1 GitHub дээрх ажиллагаа

Repository: `github.com/grl059/OS_BD1` (салбар `main`). Бүтэц нь удирдамжийн 6-р хэсгийн дагуу: `README.md` (нэр, код, зорилго, Quick Start, AI тэмдэглэл), `src/`, `tests/`, `examples/`, `docs/` (Manual, Report, screenshot, тайлангийн эх HTML). Build-ийн гаралт болон гарын үсгийн түлхүүрийг `.gitignore`-оор хассан. 9-р долоо хоногт нийтлэх хувилбарыг `v1.0` tag ба GitHub Release болгоно.

Огноо	Commit	Агуулга
2026-10-20	first commit (×2), Change1	Repository үүсгэв; эх код, жишээ, тест, анхны UI, README
2026-10-20	UI arai oiglomjtoi bolgov	UI-г энгийн хэрэглэгчид ойлгомжтой болгосон (галт тэрэг, дараалал г.м. зүйрлэл)
2026-10-25	Change2, ui	Hash, AVL, scheduler tab-уудыг UI-д нэмсэн
2026-10-25	Report zasav, Delete kds, Example	Тайлан, Manual шинэчилсэн; binary файлыг хассан
2026-10-25	Code buten bolgov	kds.c-д hash/AVL/scheduler/find; шинэ жишээ ба тест; kernel module Part I/II; README, Manual, Report засвар

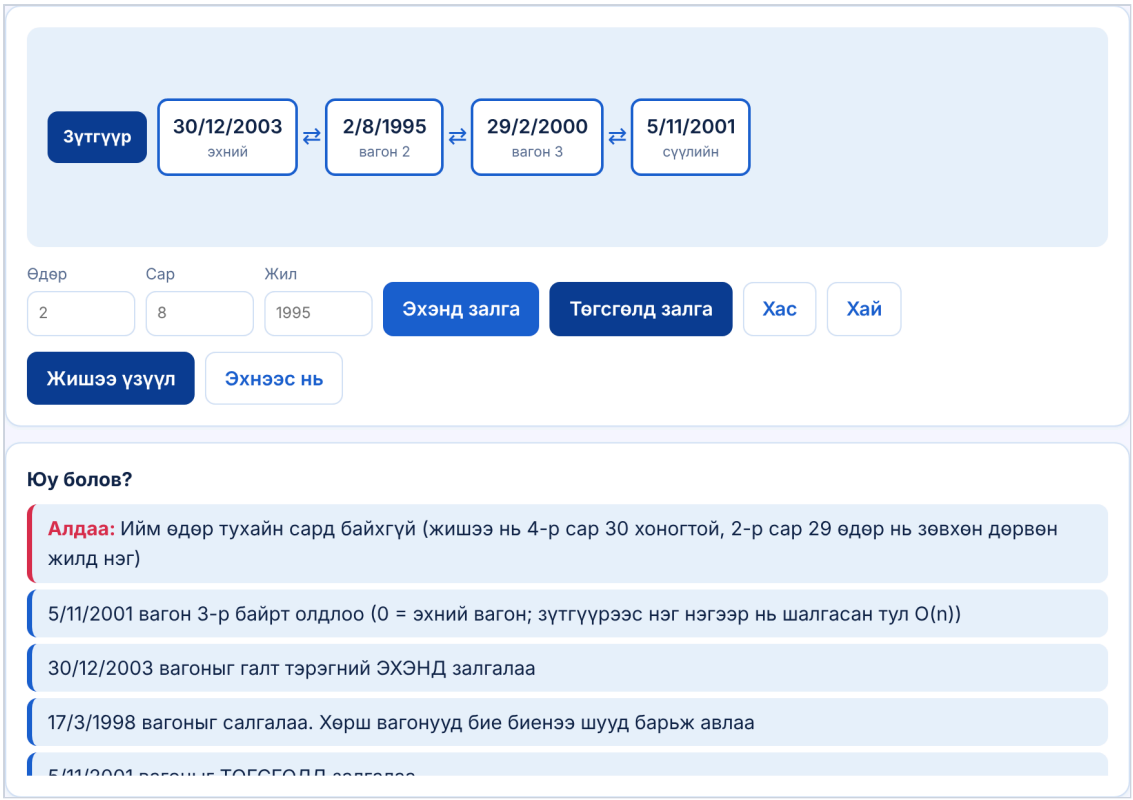
Гишүүн бүрийн хариуцсан ажил ба commit-ийг Хавсралт А-д үзүүлэв.

6. Туршилт ба UI дүрслэл

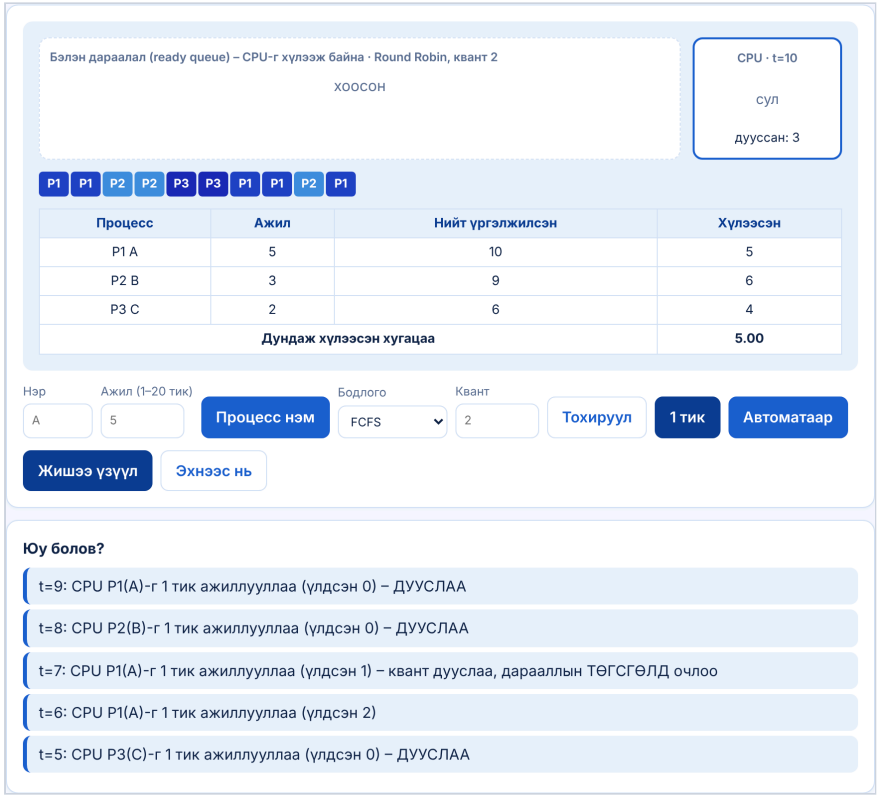
UI-д 7 tab бий: **Галт тэрэг** (list_head, ⇄ сум нь next/prev заагч), **Дэлгүүрийн дараалал** (queue), **Тавагны овоолго** (stack), **Гэрлийн самбар** (bitmap, бит бүр нэг нүд), **Хэш хүснэгт** (bucket бүрийн жагсаалт, collision), **Процесс ба CPU** (ready queue, CPU, Gantt, хүлээлтийн хүснэгт), **Хайлтын мод** (BST ба AVL, өндрийн заалттай). Бүтэц бүр дээр нэмэх, устгах, хайх товч байна.

Юуг өөрчилж турших вэ: огноо, тоо, bitmap-ийн хэмжээ (1–128), bucket-ийн тоо (1–16), түлхүүр/утга, модны төрөл (BST/AVL), процессын нэр, ажлын хугацаа (1–20), scheduler-ийн бодлого ба квант (1–10). Өгөгдлийг товч, командын мөр эсвэл script файлаар (`examples/*.txt`) оруулна. «Эхнээс нь» товч бүх бүтцийг reset хийнэ.

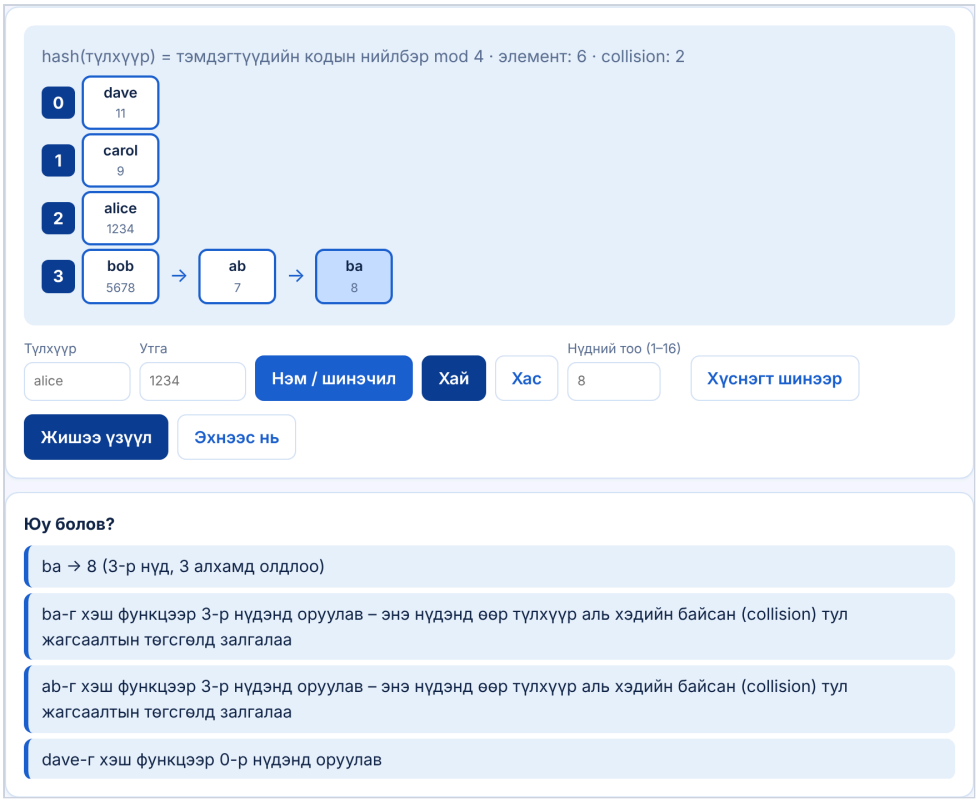
Юуг ажиглах вэ: цайвар цэнхэр дүүргэлт нь сүүлийн үйлдэлд өөрчлөгдсөн эсвэл олдсон элементийг тэмдэглэнэ. «Юу болов?» хэсэгт OK үр дүн цэнхэр зураастай, алдаа улаан зураастай «Алдаа:» гэсэн тайлбартай гарна. Хэш хүснэгтэд collision-ийн тоо болон хайлтын алхам, модонд өндөр, scheduler-т Gantt диаграм ба дундаж хүлээлтийг ажиглана.



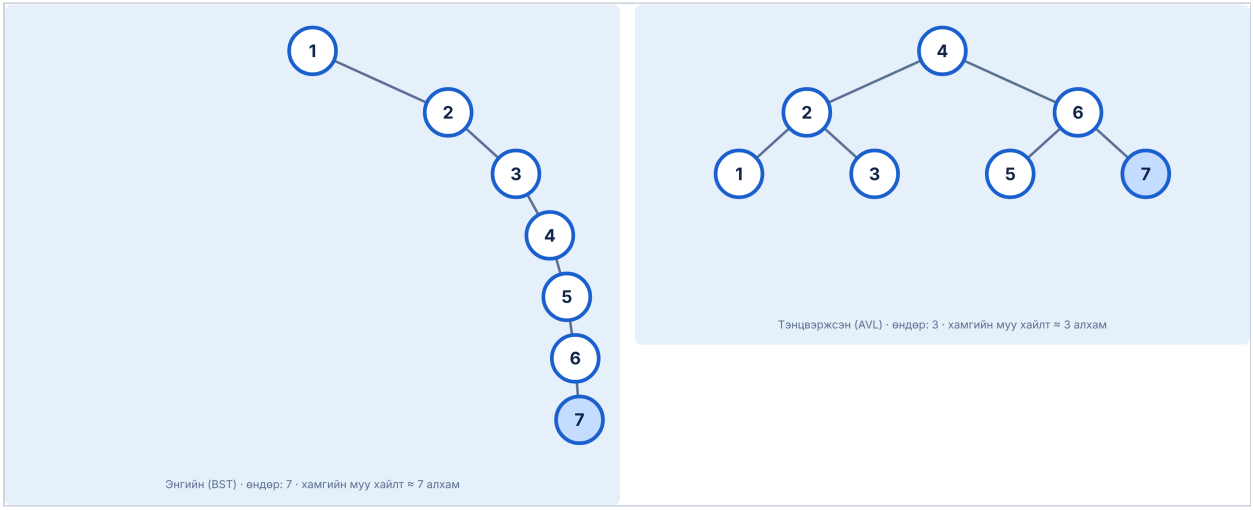
Зураг 1 – «Галт тэрэг» (list_head): 17/3/1998 вагоныг салгаж, 30/12/2003-ыг эхэнд залгасан, 5/11/2001-ийг хайж олсон; буруу огноо (31/4/2000) тайлбартай татгалзагдсан.



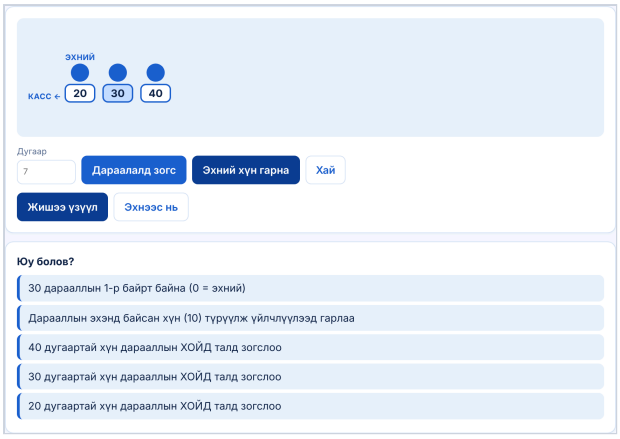
Зураг 2 – «Процесс ба CPU»: Round Robin (квант 2), A=5, B=3, C=2 тик; Gantt диаграм ба процесс бүрийн хүлээсэн хугацаа (дундаж 5.00).



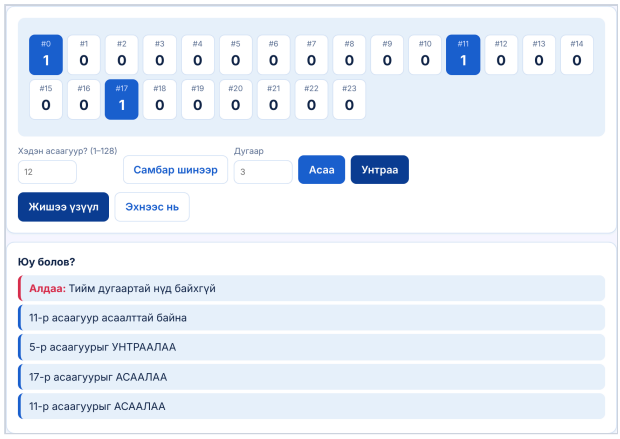
Зураг 3 – Хэш хүснэгт (4 bucket): «bob», «ab», «ba» 3-р bucket-д орж 2 collision үүссэн; «ba»-г хайхад 3 алхам шаардлаа.



Зураг 4 – Эрэмбэлсэн 1–7-г оруулсны дараа: зүүн талд энгийн BST (өндөр 7, нэг шугам), баруун талд тэнцвэржссэн AVL (өндөр 3).



Зураг 5 – Queue: 10, 20, 30, 40 нэмээд pop хийсэн, 30-г хайсан.



Зураг 6 – Bitmap (24 бит): 0, 11, 17 асаалттай, 99 индекс татгалзагдсан.

7. Туршилтын арга

Орчин: Ubuntu 24.04, gcc 13, GNU make, Node.js 22, Python 3.13, Chromium (Playwright, screenshot-д). Kernel module-д Ubuntu 24.04, kernel 6.11.0-17-generic, linux-headers.

Өгөгдөл ба хүлээгдэх үр дүн (хүлээгдэх гаралт `tests/expected_*.out`-д хадгалагдсан):

Тест	Ролт / нөхцөл	Хүлээгдэх үр дүн
textbook	examples/textbook.txt – сурах бичгийн 5 төрсөн өдөр, гүйлгэх, бүгдийг устгах	Оруулсан дарааллаар жагсаж, эцэст нь хоосон
all_structures	queue, stack, bitmap, tree	FIFO/LIFO дараалал, bitmap 1000000000000001, inorder 20, 40, 50, 70
errors	31/4, 29/2/1900, сар 13, жил 1800, хоосноос pop, size 0/129, индекс −1/99, давхардсан утга	Мөр бүр ойлгомжтой ERR, програм унахгүй
search	Бүх бүтцэд хайлт, hash collision, BST 1–5, AVL 1–7	index/position/steps, BST өндөр 5, AVL өндөр 3
scheduler	RR q=2 ба FCFS: A=5, B=3, C=2; буруу нэр, ажил, бодлого	Гараар тооцсон хуваарьтай таарна (8-р хэсэг)
scheduler_full	9 процесс үүсгэх	9 дэх нь татгалзагдана, нэг нь дууссаны дараа зөвшөөрөгдөнө
stress	5000 × (queue/stack push+pop, hash put+del, proc new+tick)	ERR 0, ASan/UBSan мэдээлэлгүй
UI == C	6 скриптийг UI-ийн ENGINE-ээр (node) ажиллуулах	Хүлээгдэх гаралттай мөр мөрөөрөө ижил
fuzz	tests/fuzz_compare.py: 8 seed × 5000 санамсаргүй команд (буруу огноо, том индекс, 8-аас олон процесс, урт түлхүүр г.м.)	C ба UI-ийн гаралт ижил
avl_check	tests/avl_check.py: 30 санамсаргүй гүйлт (50–400 insert/delete)	Inorder = эрэмбэлсэн олонлог, өндөр ≤ $1.44 \cdot \log_2(n+2)$
kernel module	src/kernel/run_module_test.sh: start=25, start=15, start=0	24 ба 18 гишүүнтэй Collatz, start=0 үед −EINVAL

Fuzz тест алдааг илрүүлж чадаж байгаа эсэхийг шалгахын тулд C-ийн AVL-ийн давхар эргүүлэлтийг зориуд устгаж туршсан. Ингэхэд seed 2 ба 4 нь модны өндрийн зөрүүг шууд илрүүлсэн тул тест үр дүнтэй болох нь батлагдсан.

8. Үр дүн

Тест	Үр дүн
textbook, all_structures, errors	**PASS ×3**
search (хайлт, hash, BST/AVL)	**PASS**
scheduler (RR, FCFS, алдаа), scheduler_full (8 процесс)	**PASS ×2**
stress (5000 × 8 команд)	**PASS, ERR 0, sanitizer мэдээлэлгүй**
UI == C (6 скрипт)	**PASS ×6**
fuzz C == UI (8 seed × 5000 команд, тэгш seed-д AVL)	**PASS ×8 (seed бүрд ≈4 700 гаралтын мөр)**
avl_check (30 гүйлт + эрэмбэлсэн 1..15)	**PASS ×32 (BST өндөр 15, AVL өндөр 4)**
Kernel module, өмнөх хувилбар (birthday, kfifo, bitmap)	**PASS: Ubuntu 24.04, kernel 6.11.0-17, insmod/rmmod (Зураг 7)**
Kernel module, Part I/II (color, Collatz)	Хэрэгжсэн; `run_module_test.sh`-ээр баталгаажуулна. Хүлээгдэх гаралтыг доор үзүүлэв.

Сурах бичгийн жишээний бодит гаралт (examples/textbook.txt)

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)   ...   OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

Хайлт, хэш хүснэгт, мод (examples/search.txt, холбогдох мөрүүд)

```
OK list find 17/3/1998 = index 1
OK list find 1/1/2001 = not found
OK queue find 20 = position 1
OK stack find 1 = position 2
OK hash put ab -> 7 (bucket=3, size=5, collision)
OK hash put ba -> 8 (bucket=3, size=6, collision)
HASH buckets=4 size=6 collisions=2: b0=[dave:11] b1=[carol:9] b2=[alice:1234] b3=
[bob:5678 -> ab:7 -> ba:8]
OK hash get ba = 8 (bucket=3, steps=3)
OK hash get bob = 5678 (bucket=3, steps=1)
OK hash get zed = not found (bucket=3)
OK hash put bob = 0 (updated, bucket=3)
ERR hash: key and value must be 1..12 letters, digits or _
TREE size=5 height=5 inorder: 1, 2, 3, 4, 5
ERR tree: mode can only be changed when the tree is empty
TREE(avl) size=7 height=3 inorder: 1, 2, 3, 4, 5, 6, 7
TREE(avl) size=6 height=3 inorder: 1, 2, 3, 5, 6, 7
```

Scheduler: гараар тооцоо ба програмын гаралт

Round Robin, квант 2; A=5, B=3, C=2 тик, бүгд t=0-д ирсэн. Тик t нь [t, t+1) хугацааны завсар тул t=5 дээр дууссан процессын дуусах хугацаа 6 болно.

Процесс	Сүүлийн тик	Дуусах хугацаа	Нийт үргэлжилсэн	Ажил	Хүлээсэн
C (P3)	5	6	6	2	4
B (P2)	8	9	9	3	6
A (P1)	9	10	10	5	5
Дундаж хүлээлт					**5.00**

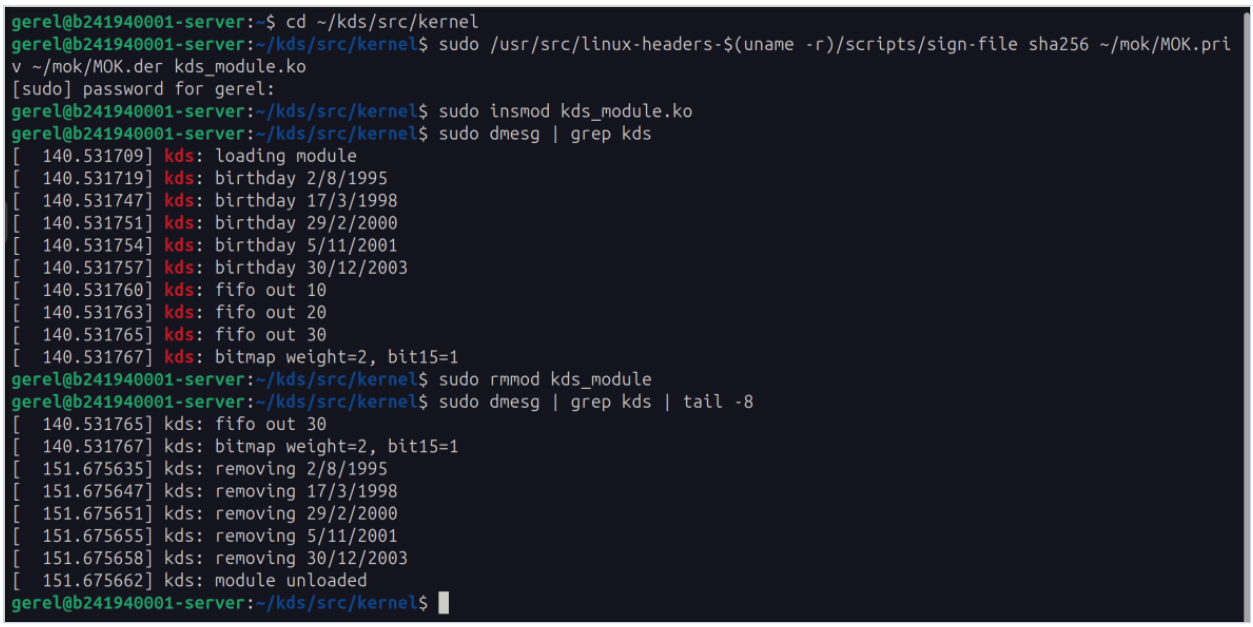
Ижил ажлыг FCFS-ээр ажиллуулахад A 0, B 5, C 8 тик хүлээж, дундаж хүлээлт 4.33 болно. Round Robin богино процесст түргэн хариу өгдөг ч энэ жишээнд дундаж хүлээлт FCFS-ээс их гарсан нь квант ба шилжилтийн өртөг юм.

```
OK sched policy rr q=2
SCHED policy=rr(q=2) time=0 running=none ready=P1(A:5), P2(B:3), P3(C:2) done=0
TICK t=0 P1(A) remaining=4
TICK t=1 P1(A) remaining=3 preempted
TICK t=2 P2(B) remaining=2
TICK t=3 P2(B) remaining=1 preempted
TICK t=4 P3(C) remaining=1
TICK t=5 P3(C) remaining=0 finished
TICK t=6 P1(A) remaining=2
TICK t=7 P1(A) remaining=1 preempted
TICK t=8 P2(B) remaining=0 finished
TICK t=9 P1(A) remaining=0 finished
TICK t=10 idle
SCHED policy=rr(q=2) time=11 running=none ready=(empty) done=3
```

Буруу оролтын бодит гаралт (examples/errors.txt)

```
ERR list: day out of range for month          (31/4/2000)
ERR list: day out of range for month          (29/2/1900 — өндөр жил биш)
ERR list: month must be 1..12
ERR list: year must be 1900..2100
ERR list: usage <op> <day> <month> <year>
ERR list: 1/1/2000 not found
ERR list: missing operation
ERR unknown structure 'frob'
ERR queue: underflow (empty)
ERR stack: underflow (empty)
ERR bitmap: size must be 1..128               (0 ба 129)
ERR bitmap: index 99 out of range 0..31
ERR tree: duplicate 5
ERR tree: 9 not found
```

Kernel module



Зураг 7 – Kernel module-ийн өмнөх хувилбарын (birthday, kfifo, bitmap) бодит ачаалалт: Ubuntu 24.04, kernel 6.11.0-17-generic, MOK гарын үсэгтэй.

Part I/II нэмэгдсэн хувилбарын хүлээгдэх dmesg гаралт (`insmod kds_module.ko start=15`). Collatz гишүүдийг хэрэглэгчийн орчинд тооцоолж шалгасан: start=25 үед 24 гишүүн, start=15 үед 18 гишүүн.

```
kds: loading module (start=15)
kds: color red=255 blue=0 green=0
kds: color red=0 blue=255 green=0
kds: color red=0 blue=0 green=255
kds: color red=128 blue=64 green=32
kds: collatz 15
kds: collatz 46   23   70   35   106   53   160   80   40   20   10   5   16   8
4   2
kds: collatz 1
kds: collatz start=15 terms=18
kds: birthday 2/8/1995 ... 30/12/2003, fifo out 10 20 30, bitmap weight=2, bit15=1
(rmmod) kds: removing color ... / removing collatz 15 ... 1 / removing birthday ...
/ module unloaded
(insmod start=0) kds: start must be 1..1000000 (got 0)   → insmod: Invalid
parameters
```

9. Хэлэлцүүлэг

Бүх буруу оролт тодорхой мессеж өгч, програм нэг ч удаа унасангүй. ASan/UBSan санах ойн алдаа, leak мэдээлээгүй. Хоёр тусдаа хэрэгжилт (C ба JavaScript) 40 000 санамсаргүй командад ижил гаралт өгсөн нь график нь бодит ажиллагааг үнэн зөв харуулж байгааг баталж байна. Эрэмбэлсэн оролтод BST-ийн өндөр n-тэй тэнцүү болж, AVL-ийнх lg n орчим байсан нь онолтой таарч байна. Хэш хүснэгтэд collision нэмэгдэх тусам хайлтын алхам өссөн нь мөн онолтой нийцнэ.

Хязгаарлалт:

1. Kernel module-ийн өмнөх хувилбарыг нэг машин дээр (kernel 6.11.0-17) ачаалж туршсан. Өөр kernel хувилбарт дахин build хийх шаардлагатай. Secure Boot идэвхтэй үед гарын үсэггүй модуль ачаалагдахгүй тул MOK түлхүүрээр гарын үсэг зурсан.
2. 1.9-д дурдсан red-black мод (`rbtree.h`) болон тусдаа singly жагсаалтыг хэрэгжүүлээгүй. Жагсаалт нь kernel-ийн list_head шиг давхар холбоост дугуй, тэнцвэржсэн модыг AVL-аар загварчилсан.
3. Hash map-ийн bucket-ийн тоо тогтмол (динамик resize байхгүй). Хэш функц нь энгийн нийлбэр тул «ab», «ba» шиг үсгийн байр солигдсон түлхүүрүүд үргэлж collision үүсгэнэ.
4. Scheduler-т бүх процесс үүсгэсэн мөчдөө ирнэ гэж үзсэн. I/O хүлээлт ба тэргүүлэх эрэмбэ байхгүй.
5. UI ба C нь тусдаа хэрэгжилт тул нэг эх кодоос үүсээгүй. Ижил байдлыг тест, fuzz-аар баталсан.
6. Queue, stack ба амьд процессын тоо 8, hash-ийн элемент 24-өөр хязгаарлагдсан. Энэ нь дүрслэлд багтаах зорилготой.

10. Дүгнэлт

Сурах бичгийн 3-р бүлгийн «Kernel Data Structures» төслийн хоёр хэсэг (color жагсаалт, `start` параметртэй Collatz жагсаалт) болон 1.9-ийн бүтэц бүрийг хэрэгжүүлсэн: холбоост жагсаалт, дараалал, стек, BST ба тэнцвэржсэн мод, хэш хүснэгт, bitmap. Нэмэх, устгах, хайх үйлдлийн дараах төлөвийг ижил командын хэлээр CLI болон график UI хоёуланд харуулж чадсан. UI дээр дараах зүйлсийг нүдээр харуулсан: энгийн мод эрэмбэлсэн оролтод нэг шугам болдог (өндөр 7) бол тэнцвэржсэн мод өндрөө 3 хэвээр хадгалдаг, хэш хүснэгтэд collision үүсэхэд хайлтын алхам нэмэгддэг. Зөв ажиллагааг 6 скрипт, stress тест, AVL-ийн инвариант шалгалт, UI==C харьцуулалт (40 000 санамсаргүй команд) болон жинхэнэ Linux дээр ачаалсан kernel module-аар нотолсон. Нэмэлтээр «Processes» сэдвийн PCB ба ready queue ашиглан FCFS ба Round Robin-ийг хүлээлтийн хугацаагаар хэмжиж харьцуулсан.

Ашигласан материал

1. A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018 – 1.9 Kernel Data Structures, 3-р бүлэг Processes ба Programming Projects.
2. Linux kernel эх код ба баримт: `include/linux/list.h`, `include/linux/kfifo.h`, `include/linux/bitmap.h`, `include/linux/moduleparam.h`; Documentation/core-api/kernel-api.rst.
3. The Linux Kernel Module Programming Guide (module_param, insmod/rmmod).
4. F.CSM302 «Бие даалт 1» удирдамж.

5. AI хэрэгсэл: Claude (Anthropic) – тайлангийн ноорог, C програмыг UI-ийн хөдөлгүүртэй нийцүүлэх, тест скрипт, баримт бичгийн засварт ашигласан. Бүх кодыг баг ажиллуулж, тестээр шалгасан.

Хавсралт А. Ажлын төлөвлөгөө ба гишүүдийн оролцоо

Талбар	Мэдээлэл
Хуваарилалтын код	4
Төсөл	Kernel Data Structures
Repository	github.com/grl059/OS_BD1
Хэл / сан / орчин	C (gcc 13), JavaScript (HTML/SVG), kernel C; Python 3, Node.js (тест); Ubuntu 24.04
Үндсэн дүрслэл	Kernel structure graph – 7 tab бүхий interactive UI

Гишүүн	Хариуцах ажил	Commit / file
Н. Ням-Од	Онолын судалгаа: сурах бичгийн 1.9 ба 3-р бүлгийн Programming Project, list_head ба container_of; тайлангийн 2–3-р хэсэг	-
А. Гэрэл	Repository удирдлага, interactive UI (docs/ui/index.html), C програм ба kernel module-ийг нэгтгэх, kernel module-ийг өөрийн Linux сервер дээр build хийж insmod/rmmod-оор турших; тайлангийн 4–5-р хэсэг	10 commit (2026-10-20…25): first commit, UI, Report zasav, Example, Code buten bolgov
А. Номин	Туршилтын өгөгдөл: examples/*.txt, хүлээгдэх гаралт, захын ба буруу оролтын тест (errors.txt); тайлангийн 7-р хэсэг	-
Э. Мягмаршагай	Процесс ба scheduler: FCFS, Round Robin-ийн хуваарийг гараар тооцож програмын гаралттай тулгах; тайлангийн 8-р хэсэг	-
О. Хүслэн	Manual (хэрэглэгчийн ба operational заавар), Manual-ын дагуу өөр компьютер дээр build/run хийж шалгах	-
Ч. Түмэншагай	Hash map ба AVL модны туршилт, UI-ийн screenshot, тайлангийн 6, 9, 10-р хэсэг	-

Тест	Оролт / нөхцөл	Хүлээгдсэн үр дүн
1	examples/textbook.txt	5 огноо дарааллаараа, устгасны дараа хоосон
2	examples/search.txt	Хайлтын индекс, hash collision 2, BST өндөр 5, AVL өндөр 3
3	examples/scheduler.txt	RR дундаж хүлээлт 5.00, FCFS 4.33
4	examples/errors.txt	Мөр бүр ERR, програм унахгүй
5	insmod kds_module.ko start=15	4 color, 18 гишүүнтэй Collatz; start=0 → татгалзана